

UNIT IV

Greedy Method

Greedy is the most straight forward design technique. Most of the problems have n inputs and require us to obtain a subset that satisfies some constraints. Any subset that satisfies these constraints is called a feasible solution. We need to find a feasible solution that either maximizes or minimizes the objective function. A feasible solution that does this is called an optimal solution.

The greedy method is a simple strategy of progressively building up a solution, one element at a time, by choosing the best possible element at each stage. At each stage, a decision is made regarding whether or not a particular input is in an optimal solution. This is done by considering the inputs in an order determined by some selection procedure. If the inclusion of the next input, into the partially constructed optimal solution will result in an infeasible solution then this input is not added to the partial solution. The selection procedure itself is based on some optimization measure. Several optimization measures are plausible for a given problem. Most of them, however, will result in algorithms that generate sub-optimal solutions. This version of greedy technique is called *subset paradigm*. Some problems like Knapsack, Job sequencing with deadlines and minimum cost spanning trees are based on *subset paradigm*.

For the problems that make decisions by considering the inputs in some order, each decision is made using an optimization criterion that can be computed using decisions already made. This version of greedy method is *ordering paradigm*. Some problems like optimal storage on tapes, optimal merge patterns and single source shortest path are based on *ordering paradigm*.

CONTROL ABSTRACTION

Algorithm Greedy (a, n)

// $a(1 : n)$ contains the „ n “ inputs

```
{
    solution :=  $\emptyset$ ; // initialize the solution to empty for  $i:=1$  to  $n$  do
    {
        x := select (a);
        if feasible (solution, x) then
            solution := Union (Solution, x);
    }return solution;
}
```

Procedure Greedy describes the essential way that a greedy based algorithm will look, once a particular problem is chosen and the functions select, feasible and union are properly implemented.

The function select selects an input from „ a “, removes it and assigns its value to „ x “. Feasible is a Boolean valued function, which determines if „ x “ can be included into the solution vector. The function Union combines „ x “ with solution and updates the objective function.

Applications

1. Job sequenced with deadlines
2. Knapsack problem
3. Huffman coding
4. Optimal merge pattern
5. Minimum cost spanning tree
6. Dijkstra's algorithm

7. Bellman ford
8. Breadth first traversal

Characteristics

- Make a sequence of choices
- Each choice is the one that seems best so, far only depends on what been so far
- Choice produces a smaller problem to be solved

JOB SEQUENCING WITH DEADLINES

When we are given a set of “n” jobs. Associated with each Job i, deadline $d_i > 0$ and profit $P_i > 0$. For any job “i” the profit p_i is earned iff the job is completed by its deadline. Only one machine is available for processing jobs. An optimal solution is the feasible solution with maximum profit.

Sort the jobs in “j” ordered by their deadlines. The array d [1 : n] is used to store the deadlines of the order of their p-values. The set of jobs j [1 : k] such that j [r], $1 \leq r \leq k$ are the jobs in „j” and $d(j[1]) \leq d(j[2]) \leq \dots \leq d(j[k])$. To test whether $J \cup \{i\}$ is feasible, we have just to insert i into J preserving the deadline ordering and then verify that $d[J[r]] \leq r$, $1 \leq r \leq k+1$

Algorithm

```

Job_sequence_with Deadline (D,J,N,K)
{
    D(0):= J(0):=0
    K:=1
    J(1):=1 //Means first job is selected
    For i=2 .....n do
    {
        R:=k
        While (D(J(R))>D(I) AND D(J(R))!= R) do
            R:=R-1
        If( D(J(R)) ≤D(I) AND D(I) > R) then
        {
            For i=k.....R+1 by -1 do
            {
                J(L+1):=J(1)
                J(R+1):=I
                K:=K+1
            }
        }
    }
}

```

Example 1:

Consider ‘N’ jobs =5 , with profits and deadlines as below

Jobs	J1	J2	J3	J4	J5
PROFIT	20	15	10	5	1
DEADLINES	2	2	1	3	3

Each and every job requires 1 unit of time for completing its job

Solution:

Consider maximum deadline of job J4 and J5 is 3

0-----1-----2-----3(unit time)

Consider the job with maximum profit first i.e., J1(20)and whose deadline is 2,it should be completed/executed before 2 units of time .

0-----1- ←J1→ -2-----3

Now consider the job with next maximum profit J2(15) and whose deadline is 2,it should be completed/executed before 2 units of time .But the time slot between 1-2 is occupied by the J1.So, if there are any time slots are empty before 1-2 consider it .

0--←J2→--1- ←J1→ -2-----3

Now consider the job with next maximum profit J3(10) and whose deadline is 1,as we know that J3 cannot be placed before timeslot 1.As it is been occupied by the J1.

So, now consider the next job with next maximum profit J4(5) and whose deadline is 3,it should be completed/executed before 3 units of time .so , it can be executed any between 0-3 time slots .As 0-1 → J2,1-2 → J1 and 2-3 → empty slots .J4 can be executed between the 2-3 timeslot.

0--←J2→--1- - ←J1→ - -2--←J4→ - -3

So the jobs which can executed are J2,J1 and J4

The profits of Jobs → J2 + J1 + J4

→ 20+15+5

→ 40

Now we need to consider the sequence of jobs which are to executed i.e., J2→J1→ J4 or J1→ J2→ J4

Tabular format solution is

JOB	DEAD LINE	TIME UNIT	SOLUTION	PROFIT
-	-	-	0	0
J1	2	[1 -2]	J1	20
J2	2	[0-1][1-2]	J1,J2	20+15
J3	1(NOT INCLUDED)	[0-1][1-2]	J1,J2	20+15
J4	3	[0-1][1-2][2-3]	J1,J2,J4	20+15+5
J5	3(NOT INCLUDED)	-	-	-

Example 2:

Consider 'N' jobs =5 , with profits and deadlines as below

Jobs	J1	J2	J3	J4	J5	J6	J7
PROFIT	35	30	25	20	15	12	5
DEADLINES	3	4	4	2	3	1	2

Each and every job requires 1 unit of time for completing its job

SOLUTION:

JOB	DEAD LINE	TIME UNIT	SOLUTION	PROFIT
-	-	-	0	0
J1	3	[2-3]	J1	35

J2	4	[2-3],[3-4]	J1,J2	35+30
J3	4	[1-2],[2-3],[3-4]	J1,J2,J3	35+30+25
J4	2	[0-1][1-2],[2-3],[3-4]	J1,J2,J3,J4	35+30+25+20
J5	-	-	-	-
J6	-	-	-	-
J7	-	-	-	-

The final profit by executing the job is 35+30+25+20=110

Knapsack problem

There are 2 versions of it

1. 0/1 knapsack problem—
 - Items are indivisible (either take an item or not)
 - Can be solved with dynamic problem (unit 3)
2. Fractional Knapsack problem
 - items are divisible (can take any fraction of an item)
 - it can be solved in greedy method

FRACTIONAL KNAPSACK PROBLEM

Let us apply the greedy method to solve the knapsack problem. We are given “n” objects and a knapsack. The object „i” has a weight W_i and the knapsack has a capacity “m”. If a fraction x_i , $0 < x_i < 1$ of object i is placed into the knapsack then a profit of $P_i X_i$ is earned. The objective is to fill the knapsack that maximizes the total profit earned.

Since the knapsack capacity is “m”, we require the total weight of all chosen objects to be at most “m”. The problem is stated as:

Constraints for weights = $\sum_{i=1}^n x_i w_i \leq m$

Constraints for profits = $Max \sum_{i=1}^n x_i p_i$

Algorithm

Initialization:

- Sort the n objects from large to small based on their ratios is values /weights = V_i/W_i
- We assume the array $W[1....n]$ and $V[1....n]$ store the respective weights & values after sorting
- Initialize array $X[1...n]$ to zero
- Weights =0, i=1

While (i<=n and weights <w) do

{

 If (weights +w[i] <=w) then

$X[i]=1$

 else

$x[i]=(W-\text{weights}/w[i])$

 weights =weights[i]*w[i]

 i++;

```

    }
Greedy_fractional_knapsack(W[1...n],p[1...n],w)
{
    For i=1 to n
        Do x[i]=0
    Weight=0
    For l = 1 to n
        If weight +w[i] <= W then
            X[i]=1
            Weight=weight[i]
        Else
            X[i]=(W-weight)/w[i]
            Weight=w
        Break;
    Return x
}

```

Example Problem 1:

Consider the following instance of the knapsack problem: $n = 7$, m or $w = 15$, $(p_1, p_2, p_3, p_4, p_5, p_6, p_7) = (10, 5, 15, 7, 6, 18, 3)$ and $(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1)$.

Solution:

First we will calculate the profits for the individual weights i.e., p/w

Pi	P	Wi	W	p/w	Xi
P1	10	W1	2	5	X1
P2	5	W2	3	1.6	X2
P3	15	W3	5	3	X3
P4	7	W4	7	1	X4
P5	6	W5	1	6	X5
P6	18	W6	4	4.5	X6
P7	3	W7	1	3	X7

The fraction of individual weights which needed to be considered for placing them into the knapsack i.e., $x_1, x_2, \dots, x_7$

→ We need to consider the object which has the highest profit from the above table i.e., $6 = x_5$

As it is included in the knapsack the value of $X_5 = 1$

Now the weight becomes W or $M = 15 - w_5 = 15 - 1 = 14$

→ Similarly consider the next highest profit from the above table i.e., $5 = x_1$

As it is included in the knapsack the value of $X_1 = 1$

Now the weight becomes W or $M = 14 - w_1 = 14 - 2 = 12$

→ Similarly consider the next highest profit from the above table i.e., $4.5 = x_6$

As it is included in the knapsack the value of $X_6 = 1$

Now the weight becomes W or $M = 12 - w_6 = 12 - 4 = 8$

As X_3 and X_7 profits are same then we will consider the x_3 (weight=5) first then x_7 (weight=1) later

→ Similarly consider the next highest profit from the above table i.e., $3 = x_3$

As it is included in the knapsack the value of $X_3=1$

$$\text{Now the weight becomes } W \text{ or } M = 8 - w_3 = 8 - 5 = 3$$

→ Similarly consider the next highest profit from the above table i.e. $3 = x_7$

As it is included in the knapsack the value of $X_7=1$

$$\text{Now the weight becomes } W \text{ or } M = 3 - w_7 = 3 - 1 = 2$$

→ Similarly consider the next highest profit from the above table i.e. $1.6 = x_2$

As it is included in the knapsack the value of $X_2=1$

****In the knapsack we cannot place entire 3 units of weight, so we consider only 2 units of weight.**

So, the value of X_2 will be the form of fraction i.e., $2/3$

$$X_2 = 2/3$$

$$\text{Now the weight becomes } W \text{ or } M = 2 - w_2 = 2 - 2 = 0$$

As x_4 is not included in the knapsack its value will be 0, $x_4=0$

The order if considering the objects is $X=(1, 2/3, 1, 0, 1, 1, 1)$ as $0 \leq x \leq 1$

Constraints for weights = $\sum_{i=1}^n x_i w_i \leq m$

$$\Rightarrow 1*2 + (2/3)*3 + 1*5 + 0*7 + 1*1 + 1*4 + 1*1 \leq 15$$

$$\Rightarrow 2 + 2 + 5 + 0 + 1 + 4 + 1 \leq 15$$

$$\Rightarrow 15 \leq 15$$

Constraints for profits = $\text{Max } \sum_{i=1}^n x_i p_i$

$$\Rightarrow 1*10 + (2/3)*5 + 1*15 + 0*7 + 1*6 + 1*18 + 1*3$$

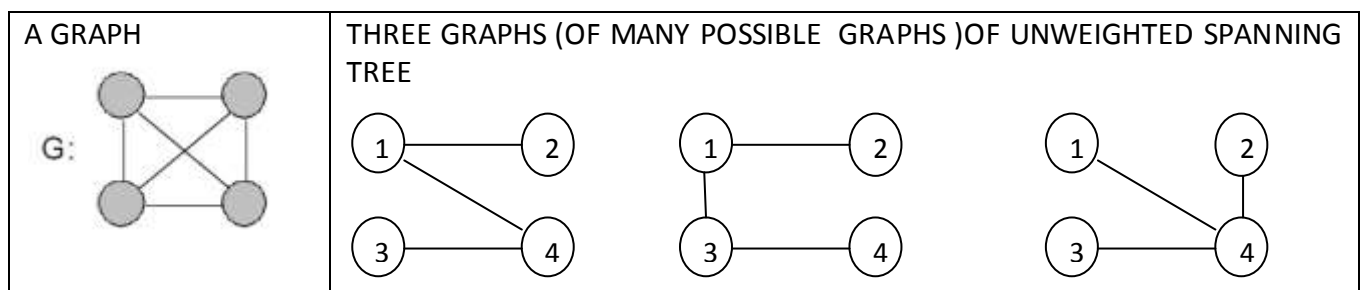
$$\Rightarrow 10 + 3.33 + 15 + 0 + 6 + 18 + 3$$

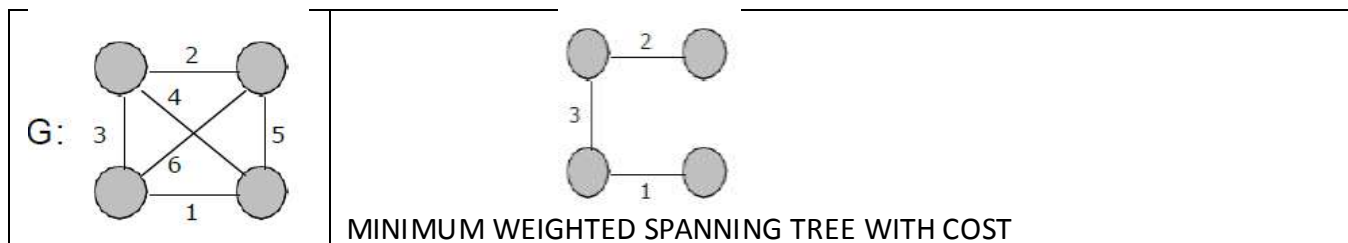
$$\Rightarrow 55.33$$

MINIMUM COST SPANNING TREE

A spanning tree for a connected graph is a tree whose vertex set is the same as the vertex set of the given graph, and whose edge set is a subset of the edge set of the given graph. i.e., any connected graph will have a spanning tree.

Weight of a spanning tree $w(T)$ is the sum of weights of all edges in T . The Minimum spanning tree (MST) is a spanning tree with the smallest possible weight.





MINIMUM WEIGHTED SPANNING TREE WITH COST

3. Let $G(V, E)$ be an undirected connected graph with vertices ' v ' and edge ' E '.
4. A sub graph $t=(V, E)$ of the graph G is a spanning tree of G iff ' t ' is a tree.
5. The problem is to generate a graph $G=(V, E)$ where ' E ' is the subset of E . G is the minimum spanning tree
6. Each and every edge will contain the given non-negative length, connect all the nodes with edge present in set ' E ' and weights has to be minimum

Definition

- A spanning tree of a graph is an undirected tree consisting of only that edge that is necessary to connect all the vertices in the original graph.
- A spanning tree has a property that for any pair of vertices there exists only one path between and the insertion of an edge to a spanning tree form a unique cycle

APPLICATIONS

- Analysis for electrical circuits
- Shortest route problems

To explain how to find a Minimum Spanning Tree, we will look at two algorithms: the Kruskal algorithm and the Prim algorithm. Both algorithms differ in their methodology, but both eventually end up with the MST. Kruskal's algorithm uses edges, and Prim's algorithm uses vertex connections in determining the MST.

a. KRUSKAL'S ALGORITHM

In this the selection function chooses edges in increasing order of length without worrying too much about their connection to previously chosen edge, except that never to form a cycle. The result is to form a forest of tree that grows until all the trees in a forest merge in a single tree

- In this algorithm a minimum cost –spanning tree ' T ' is built edge by edge
- Edges are considered for inclusion in ' T ' in increasing order of their cost.
- An edge is included in ' T ' if it doesn't form acyclic with edges already in T
- To find the minimum cost spanning tree the edges are inserted to tree in increasing order of their cost

Algorithm:

The algorithm for finding the MST, using the Kruskal's method is as follows:

Algorithm Kruskal ($E, cost, n, t$)

// E is the set of edges in G . G has n vertices. $cost [u, v]$ is the
 // cost of edge (u, v) . ' t ' is the set of edges in the minimum-cost spanning tree.
 // The final cost is returned.
 {

Construct a heap out of the edge costs using heapify; for $i := 1$ to n do parent

```

[i] := -1; // Each vertex is in a different set.
i := 0; mincost := 0.0;
while ((i < n - 1) and (heap not empty)) do
{
    Delete a minimum cost edge (u, v) from the
    heap and re-heapify using Adjust;
    j := Find (u); k :=
    Find (v); if (j ≠ k)
    then
    {
        i := i + 1;
        t [i, 1] := u; t [i, 2] :=
        v; mincost := mincost +
        cost [u, v]; Union (j,
        k);
    }
}
if (i ≠ n-1) then write ("no
spanning tree"); else return
mincost;
}

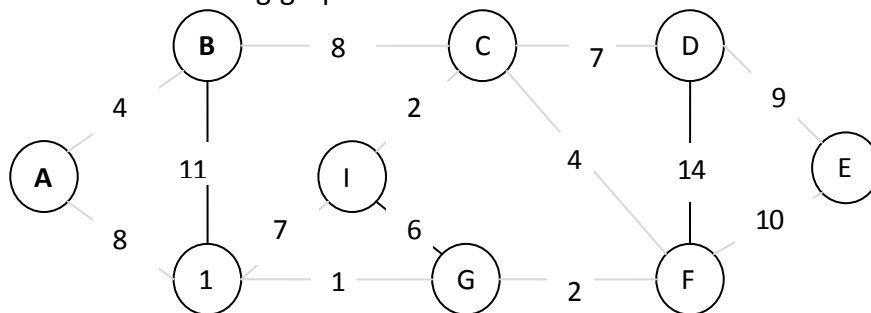
```

Running time:

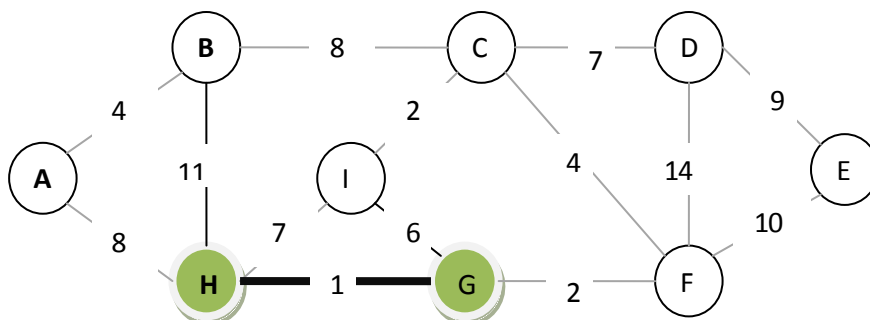
- The number of finds is at most $2e$, and the number of unions at most $n-1$. Including the initialization time for the trees, this part of the algorithm has a complexity that is just slightly more than $O(n + e)$.
- We can add at most $n-1$ edges to tree T . So, the total time for operations on T is $O(n)$.
- Summing up the various components of the computing times, we get $O(n + e \log e)$ as asymptotic complexity

Example

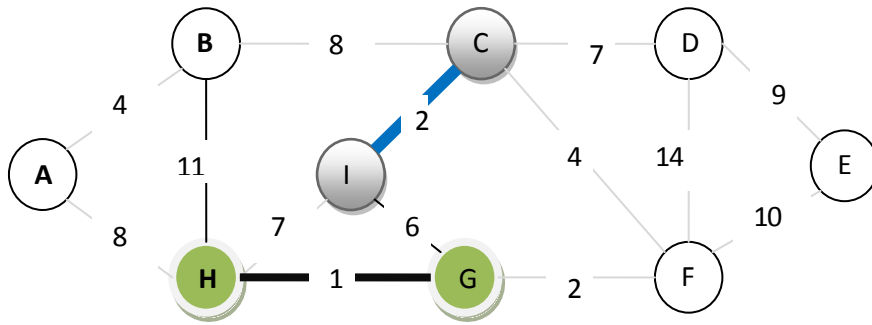
Consider the following graph



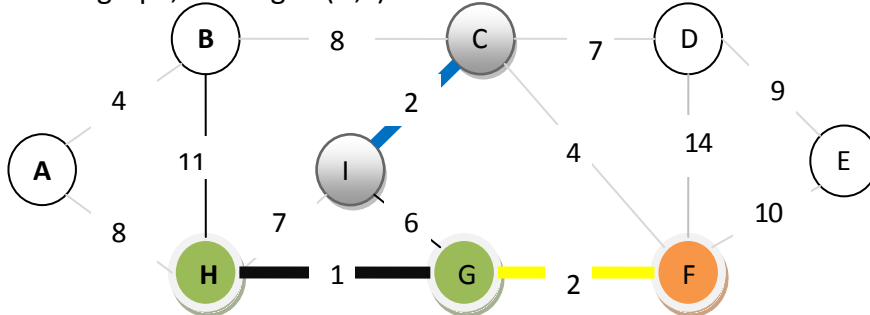
STEP 1 from the graph, the Edges (g,h) is the shortest . As it is the minimum path



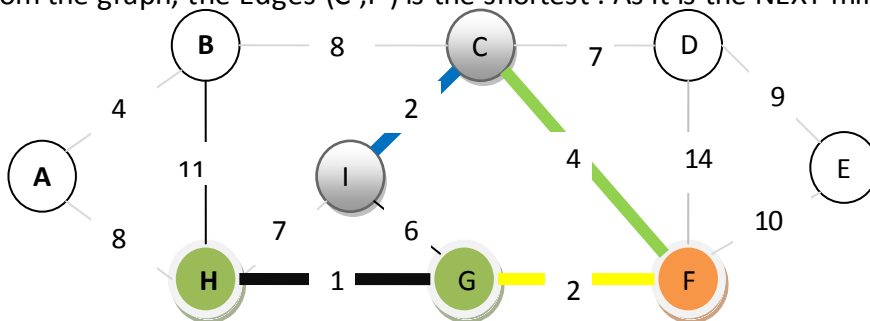
STEP 2 from the graph, the Edges (I,C) is the shortest . As it is the minimum path



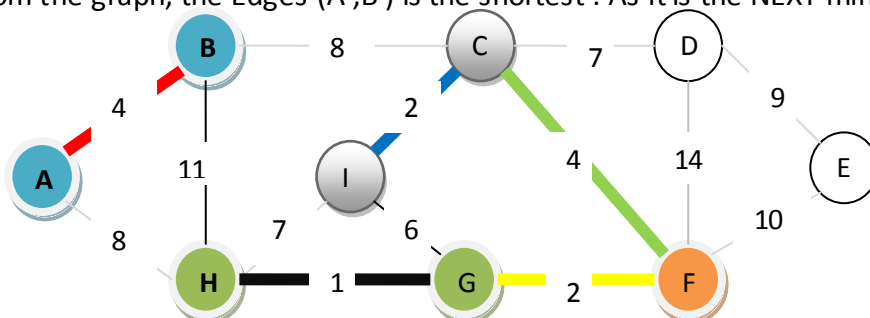
STEP 3 from the graph, the Edges (G,F) is the shortest . As it is the NEXT minimum path



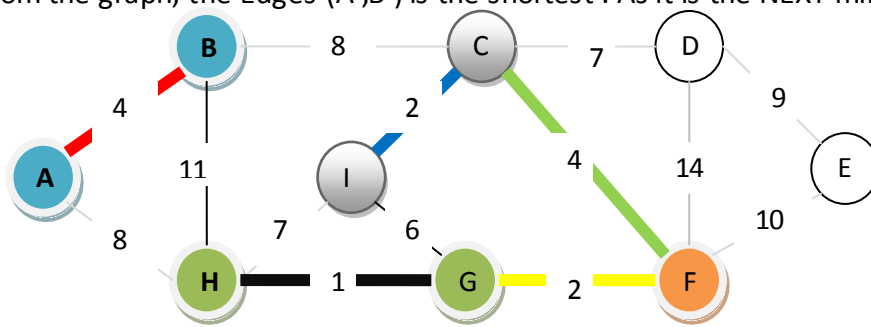
STEP 4 from the graph, the Edges (C ,F) is the shortest . As it is the NEXT minimum path



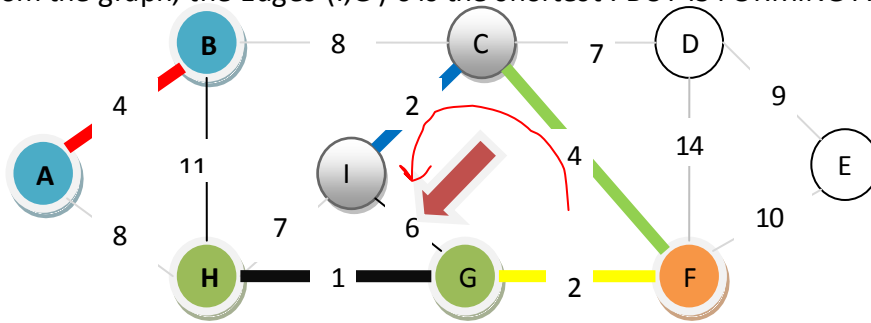
STEP 5 from the graph, the Edges (A ,B) is the shortest . As it is the NEXT minimum path



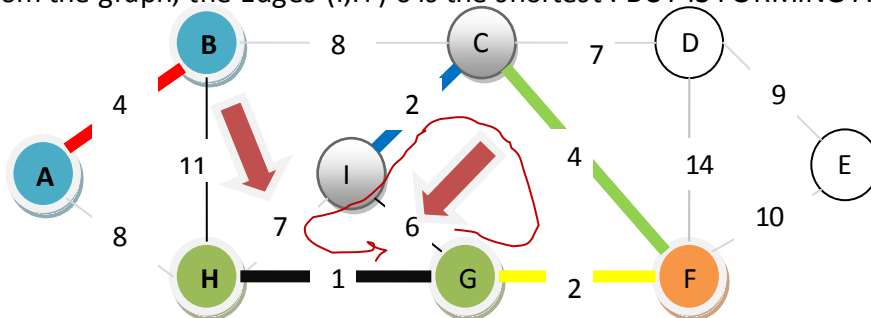
STEP 6 from the graph, the Edges (A,B) is the shortest . As it is the NEXT minimum path



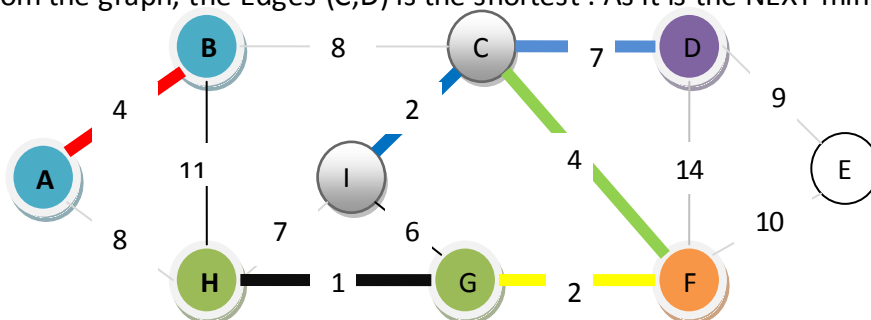
STEP 7 from the graph, the Edges (I,G) 6 is the shortest . BUT IS FORMING A CYCLE (C,I,G,H)



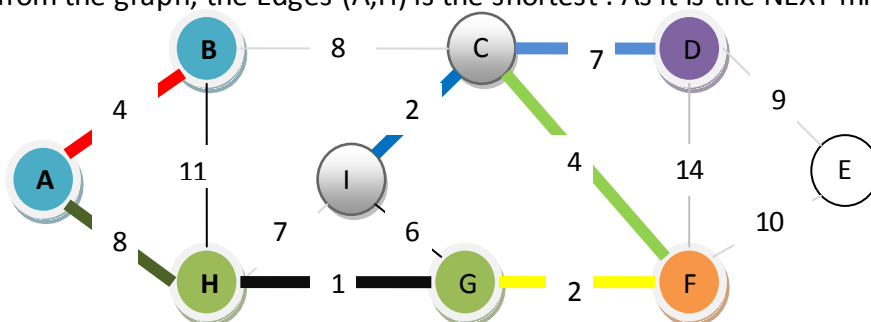
STEP 8 from the graph, the Edges (I,H) 6 is the shortest . BUT IS FORMING A CYCLE (I,G,H)



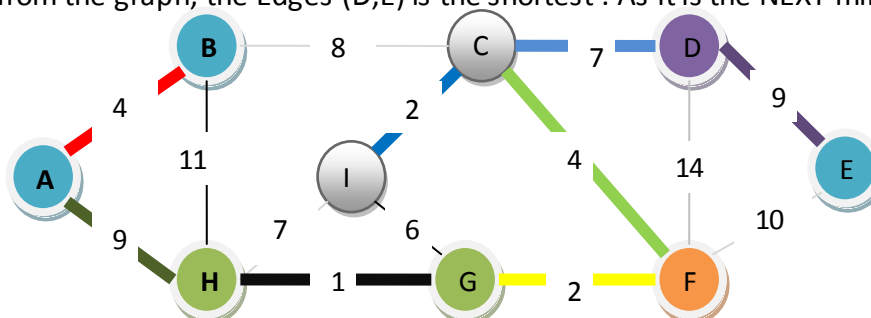
STEP 9 from the graph, the Edges (C,D) is the shortest . As it is the NEXT minimum path



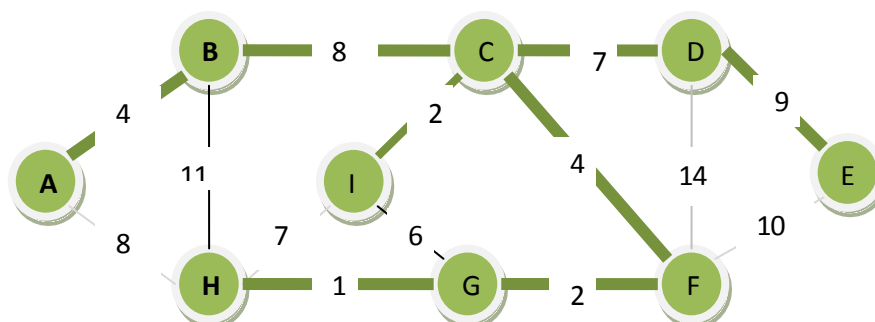
STEP 10 from the graph, the Edges (A,H) is the shortest . As it is the NEXT minimum path



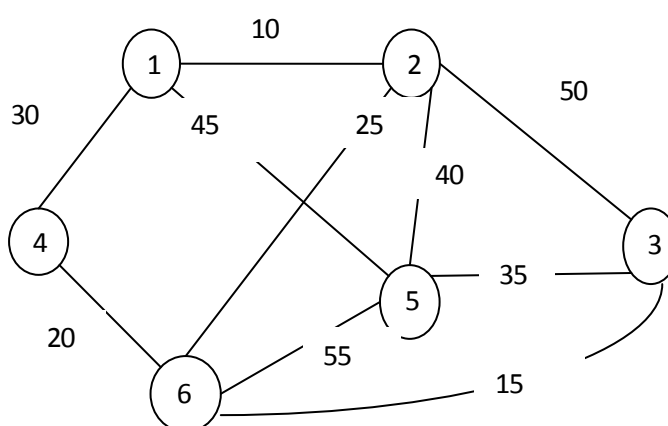
STEP 11 from the graph, the Edges (D,E) is the shortest . As it is the NEXT minimum path



OTHER SOLUTIONS



Example 2

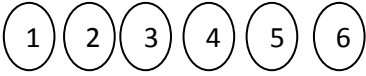
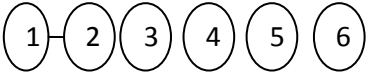
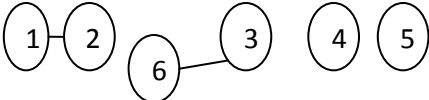
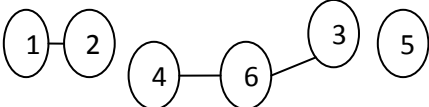
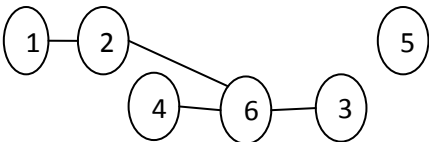
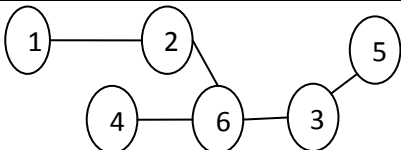


Solutions

Cost	10	15	20	25	30	35	40	45	50	55
Edge	(1, 2)	(3, 6)	(4, 6)	(2, 6)	(1, 4)	(3, 5)	(2, 5)	(1, 5)	(2, 3)	(5, 6)

The edge set T together with the vertices of G defines a graph that has up to n connected components. Let us represent each component by a set of vertices in it. These vertex sets are disjoint. To determine whether the edge (u, v) creates a cycle, we need to check whether u and v are

in the same vertex set. If so, then a cycle is created. If not then no cycle is created. Hence two finds on the vertex sets suffice. When an edge is included in T, two components are combined into one and a union is to be performed on the two sets.

EDGE	COST	SPANNING FOREST	EDGE SETS	REMARKS
			$\{1\}, \{2\}, \{3\}, \{4\}, \{5\}, \{6\}$	
(1,2)	10		$\{1,2\}, \{3\}, \{4\}, \{5\}, \{6\}$	The vertices 1 and 2 are in different sets ,so the edges are combined
(3,6)	15		$\{1,2\}, \{3,6\}, \{4\}, \{5\}$	The vertices 3 and 6 are in different sets ,so the edges are combined
(4,6)	20		$\{1,2\}, \{3,4,6\}, \{5\}$	The vertices 4 and 6 are in different sets ,so the edges are combined
(2,6)	25		$\{1,2,3,4,6\}, \{5\}$	The vertices 2 and 6 are in different sets ,so the edges are combined
(1,4)	30	Reject		The vertices 1 and 4 are in same sets ,so the edges are not combined
(3,5)	35		$\{1,2,3,4,5,6\}$	The vertices 3 and 5 are different set ,so ,they are combined,

PRIM'S ALGORITHM

A given graph can have many spanning trees. From these many spanning trees, we have to select a cheapest one. This tree is called as minimal cost spanningtree.

Minimal cost spanning tree is a connected undirected graph G in which each edge is labeled with a number (edge labels may signify lengths, weights other than costs). Minimal cost spanning tree is a spanning tree for which the sum of the edge labels is as small as possible

The slight modification of the spanning tree algorithm yields a very simple algorithm for finding an MST. In the spanning tree algorithm, any vertex not in the tree but connected to it by an edge can be added. To find a Minimal cost spanning tree, we must be selective - we must always add a new vertex for which the cost of the new edge is as small as possible.

This simple modified algorithm of spanning tree is called prim's algorithm for finding an Minimal cost spanning tree.

Prim's algorithm is an example of a greedy algorithm

Algorithm *Algorithm Prim (E, cost, n, t)*

// E is the set of edges in G. cost [1:n, 1:n] is the cost
 // adjacency matrix of an n vertex graph such that cost [i, j] is
 // either a positive real number or ∞ if no edge (i, j) exists.
 // A minimum spanning tree is computed and stored as a set of
 // edges in the array t [1:n-1, 1:2]. (t [i, 1], t [i, 2]) is an edge in

```

// the minimum-cost spanning tree. The final cost is returned.
{
    Let (k, l) be an edge of minimum cost in E;
    mincost := cost [k, l];
    t [1, 1] := k; t [1, 2] := l;
    for i := 1 to n do // Initialize near
        if (cost [i, l] < cost [i, k]) then
            near [i] := l;
        else
            near [i] := k;
            near [k] := near [l] := 0;
    for i := 2 to n - 1 do // Find n - 2 additional edges for t.
    {
        Let j be an index such that near [j] ≠ 0 and cost [j, near [j]] is minimum;
        t [i, 1] := j;
        t [i, 2] := near [j];
        mincost := mincost + cost [j, near [j]];
        near [j] := 0
        for k := 1 to n do // Update near[].
            if ((near [k] ≠ 0) and (cost [k, near [k]] > cost [k, j]))
                then near [k] := j;
    }
    return mincost;
}

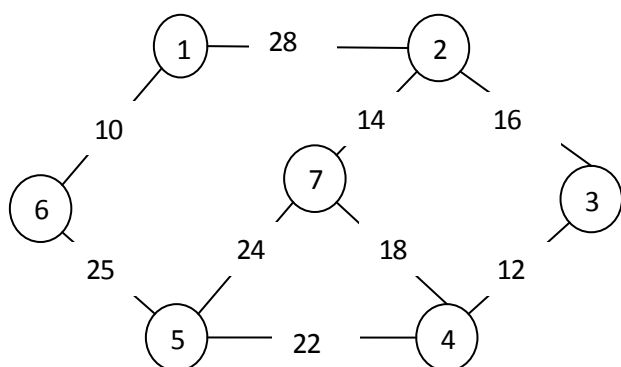
```

Running time:

We do the same set of operations with dist as in Dijkstra's algorithm (initialize structure, m times decrease value, n - 1 times select minimum). Therefore, we get $O(n^2)$ time when we implement dist with array, $O(n + |E| \log n)$ when we implement it with a heap.

For each vertex u in the graph we dequeue it and check all its neighbors in $O(1 + \deg(u))$ time.

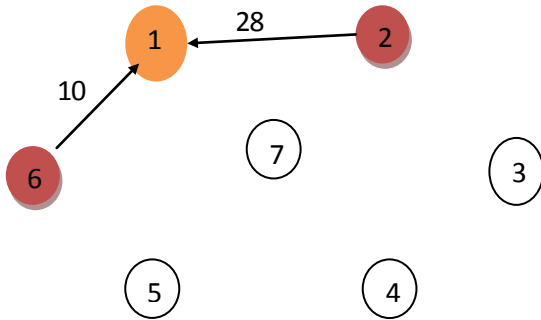
Example 1:



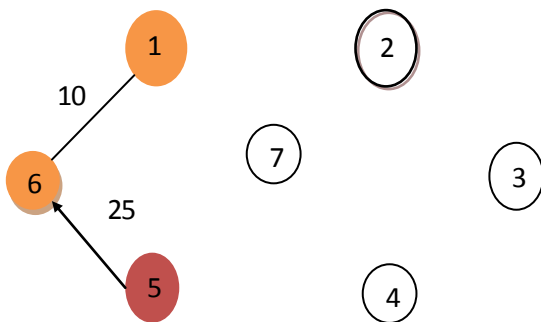
Solution: The cost adjacency matrix is

0	28	∞	∞	∞	10	∞
28	0	16	∞	∞	∞	14
∞	16	0	12	∞	∞	∞
∞	∞	12	0	22	∞	18
∞	∞	∞	22	0	15	24
10	∞	∞	∞	25	0	∞
∞	14	∞	18	24	∞	0

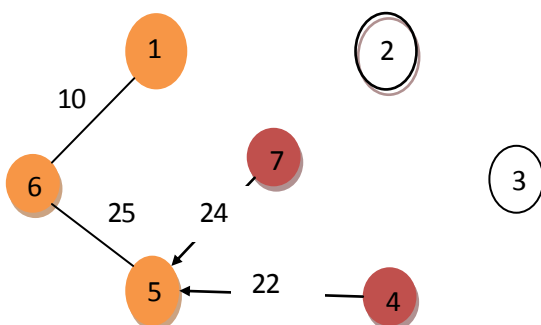
The stepwise progress of the prim's algorithm is as follows:

Step 1

Vertex	1	2	3	4	5	6	7
Status	0	1	1	1	1	1	1
Dist.	0	28	∞	∞	∞	10	∞
Next	*	1	1	1	1	1	1

STEP 2

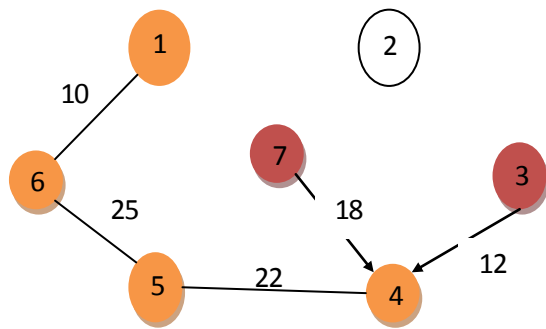
Vertex	1	2	3	4	5	6	7
Status	0	1	1	1	1	0	1
Dist.	0	∞	∞	∞	25	10	∞
Next	*	1	1	1	6	1	1

STEP 3

Vertex	1	2	3	4	5	6	7
Status	0	1	1	1	0	0	1
Dist.	0	∞	∞	22	25	10	24
Next	*	1	1	5	6	1	5

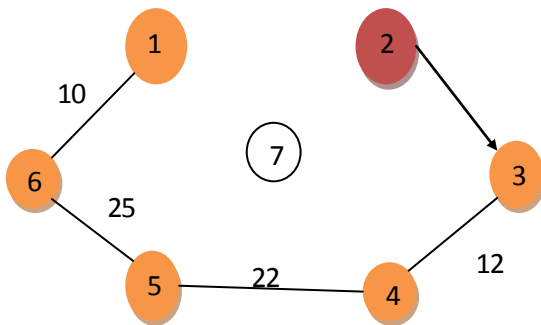
STEP 4

Vertex	1	2	3	4	5	6	7
Status	0	1	1	0	0	0	1

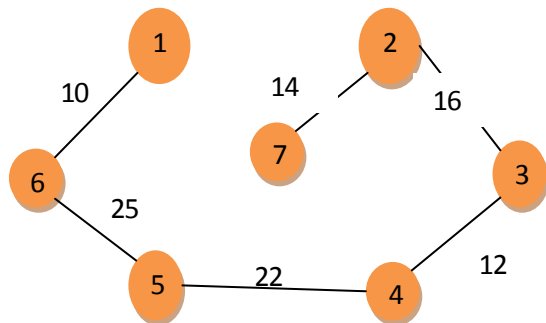


**Dist.
Next**

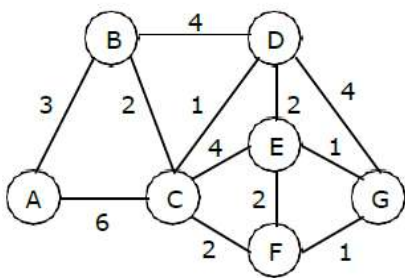
0	∞	12	22	25	10	18
*	1	4	5	6	1	4

STEP 5

Vertex	1	2	3	4	5	6	7
Status	0	1	0	0	0	0	1
Dist.	0	16	12	22	25	10	∞
Next	*	3	4	5	6	1	1

STEP 6

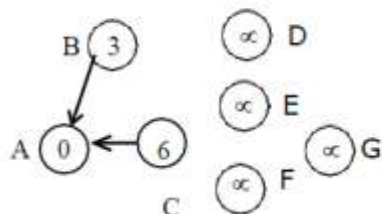
Vertex	1	2	3	4	5	6	7
Status	0	0	0	0	0	0	0
Dist.	0	16	12	22	25	10	14
Next	*	3	4	5	6	1	2

Example 2

The cost adjacency matrix is

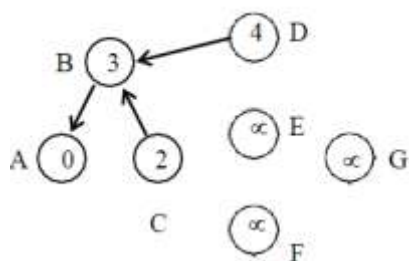
	A	B	C	D	E	F	G
A	0	3	6	∞	∞	∞	∞
B	3	0	2	4	∞	∞	∞
C	6	2	0	1	4	2	∞
D	∞	4	1	0	2	∞	4
E	∞	∞	4	2	0	2	1
F	∞	∞	2	∞	2	0	1
G	∞	∞	∞	4	1	1	0

Step 1:



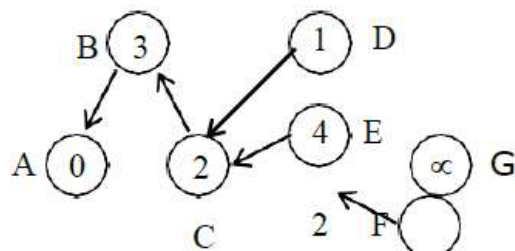
Vertex	A	B	C	D	E	F	G
Status	0	1	1	1	1	1	1
Dist.	0	3	6	∞	∞	∞	∞
Next	*	A	A	A	A	A	A

Step 2:



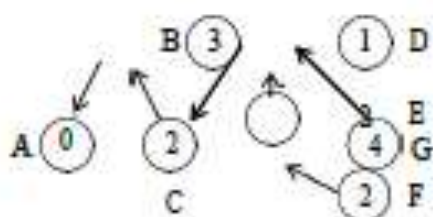
Vertex	A	B	C	D	E	F	G
Status	0	0	1	1	1	1	1
Dist.	0	3	2	4	∞	∞	∞
Next	*	A	B	B	A	A	A

Step 3:



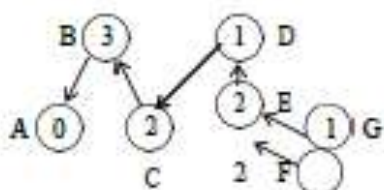
Vertex	A	B	C	D	E	F	G
Status	0	0	0	1	1	1	1
Dist.	0	3	2	1	4	2	∞
Next	*	A	B	C	C	C	A

Step 4:



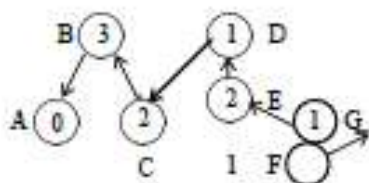
Vertex	A	B	C	D	E	F	G
Status	0	0	0	0	1	1	1
Dist.	0	3	2	1	2	2	4
Next	*	A	B	C	D	C	D

Step 5:



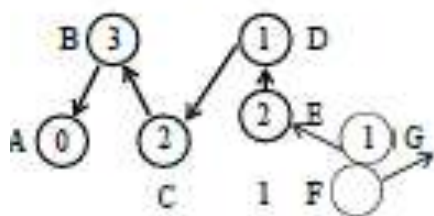
Vertex	A	B	C	D	E	F	G
Status	0	0	0	0	1	0	1
Dist.	0	3	2	1	2	2	1
Next	*	A	B	C	D	C	E

Step 6:



Vertex	A	B	C	D	E	F	G
Status	0	0	0	0	0	1	0
Dist.	0	3	2	1	2	1	1
Next	*	A	B	C	D	G	E

Step 7:



Vertex	A	B	C	D	E	F	G
Status	0	0	0	0	0	0	0
Dist.	0	3	2	1	2	1	1
Next	*	A	B	C	D	G	E

SINGLE SOURCE SHORTEST PATH PROBLEM

In the previously studied graphs, the edge labels are called as costs, but here we think them as lengths. In a labeled graph, the length of the path is defined to be the sum of the lengths of its edges.

In the single source, all destinations, shortest path problem, we must find a shortest path from a given source vertex to each of the vertices (called destinations) in the graph to which there is a path.

Algorithm:

Algorithm Shortest-Paths (v, cost, dist, n)

```
// dist [j],  $1 \leq j \leq n$ , is set to the length of the shortest path
// from vertex v to vertex j in the digraph G with n vertices.
// dist [v] is set to zero. G is represented by its
// cost adjacency matrix cost [1:n, 1:n].
{
```

```
  for i := 1 to n do
```

```
  {
```

```
    S [i] := false; // Initialize S.
```

```
    dist [i] := cost [v, i];
```

```
  }
```

```
  S[v] := true;
```

```
  dist[v] := 0.0; // Put v in S.
```

```
  for num := 2 to n - 1 do
```

```
  {
```

```
    //Determine n - 1 paths from v.
```

```

//Choose u from among those vertices not in S such that dist[u] is minimum;
S[u] := true;                                // Put u in S.
for (each w adjacent to u with S [w] = false) do
    if (dist [w] > (dist [u] + cost [u, w])) then    // Update distances
        dist [w] := dist [u] + cost [u, w];
    }
}

```

Running time:

- the minimum possible time for such an algorithm would be $\Omega(|E|)$
- the cost of adjacency matrix takes $O(n^2)$
- the overall time complexity is **$O((n+|E|)\log n)$**

Example 1

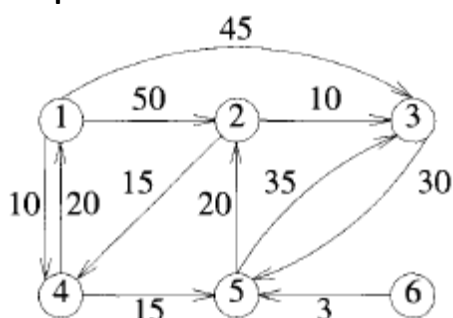


figure A

Shortest path from 1 (table 1)

Path	length
1 -> 4	10
1->4->5	10+15=25
1->4->5->2	10+15+20=45
1->3	45

Shortest path from 2 (table 2)

Path	length
2 -> 3	10
2->4	15
2->4->1	15+20=35
2->4->5	15+15=30

Shortest path from 3 (table 3)

Path	length
3 -> 5	30
5->5->2	30+20=50
3->5->2->4	30+20+15=65
3->5->2->4->1	30+20+15+20=85

Shortest path from 4 (table 4)

Path	length
4 -> 1	20
4->1->2	20+50=70
4->1->3	20+45=65
4->5	15

Shortest path from 5 (table 5)

Path	Length
5->3	35
5->2	20
5->2->4	20+15=35
5->2->4->1	20+15+20=55

From fig A:

Step 1: The adjacency matrix can be

$$\begin{bmatrix} - & 50 & 45 & 10 & \infty & \infty \\ \infty & - & 10 & 15 & \infty & \infty \\ \infty & \infty & - & \infty & 30 & \infty \\ 20 & \infty & \infty & - & 15 & \infty \\ \infty & 15 & 35 & \infty & - & \infty \\ \infty & \infty & \infty & \infty & 3 & - \end{bmatrix}$$

Step 2: Consider 1 to be the source and choose the minimum entry in the row of 1. From the above matrix the minimum in row 1 is 10

$$\begin{bmatrix} - & 50 & 45 & 10 & \infty & \infty \\ \infty & - & 10 & 15 & \infty & \infty \\ \infty & \infty & - & \infty & 30 & \infty \\ 20 & \infty & \infty & - & 15 & \infty \\ \infty & 15 & 35 & \infty & - & \infty \\ \infty & \infty & \infty & \infty & 3 & - \end{bmatrix}$$

Step 3 : find the column in which it is present, i.e., is 4, hence the node next to be visited is 4

Step 4: compute a matrix by eliminating 1 and 4 columns. Initially retain only 1, the second row is computed by adding 10 to all rows of 4

next node	2	3	5	6
1->next node	50	45	∞	∞
1→4→next node	10+ ∞	10+ ∞	10+15	10+ ∞

Step 5: find the minimum in column .Now select the minimum from the resulting row . From the above we get 25(10+15).Repeat step 3 followed by step 4 till all the vertices are covered or single column is left

	2	3	5	6
1->next node	50	45	∞	∞
1→4→next node	10+ ∞	10+ ∞	10+15	10+ ∞

Step 6

	2	3	6
1->next node	50	45	∞
1→4→5->next node	25+20	25+35	25+ ∞

Step 7

	3	6
1->next node	45	∞
1→4→5->2->next node	45+10	45 + ∞

The cheapest path from 1 to all other given vertices is as follows 1->4->5->2->3

PREVIOUS QUESTIONS**0/1 knapsack problem**

1. Can we solve 0/1 knapsack problem with greedy method? Comment on your answer
2. Discuss about fractional knapsack problem .Consider the following instances of knapsack problem with $n=3$. $m=20$ profits $(p_1,p_2,p_3)=(25,24,15)$ and weights $(w_1,w_2,w_3)=(18,15,10)$
Obtain the optimal solution using greedy method
3. **Find an optimal solution to the knapsack instance $n=4$ objects and the capacity of knapsack $m=15$,profits $(10,5,7,11)$ and weights are $(3,4,3,5)$**
4. **Find an optimal solution to the knapsack instance $n=4$ objects and the capacity of knapsack $m=15$, profits $(10, 5, 7, 11)$ and weight are $(3, 4, 3, 5)$.**
5. **What is greedy strategy?**
6. **Discuss briefly about the minimum cost spanning tree.**

job sequencing

1. Explain the greedy technique for solving the job sequencing problem
2. **State the Job – Sequencing with deadlines problem. Find an optimal sequence to the $n = 5$ Jobs where profits $(P_1, P_2, P_3, P_4, P_5) = (20, 15, 10, 5, 1)$ and deadlines $(d_1, d_2, d_3, d_4, d_5) = (2, 2, 1, 3, 3)$.**
3. State the Job – Sequencing Deadline Problem.
4. Write an algorithm for job sequencing with deadline .
5. State the job –sequencing deadline problem
6. Explain the greedy technique for solving the Job Sequencing problem.